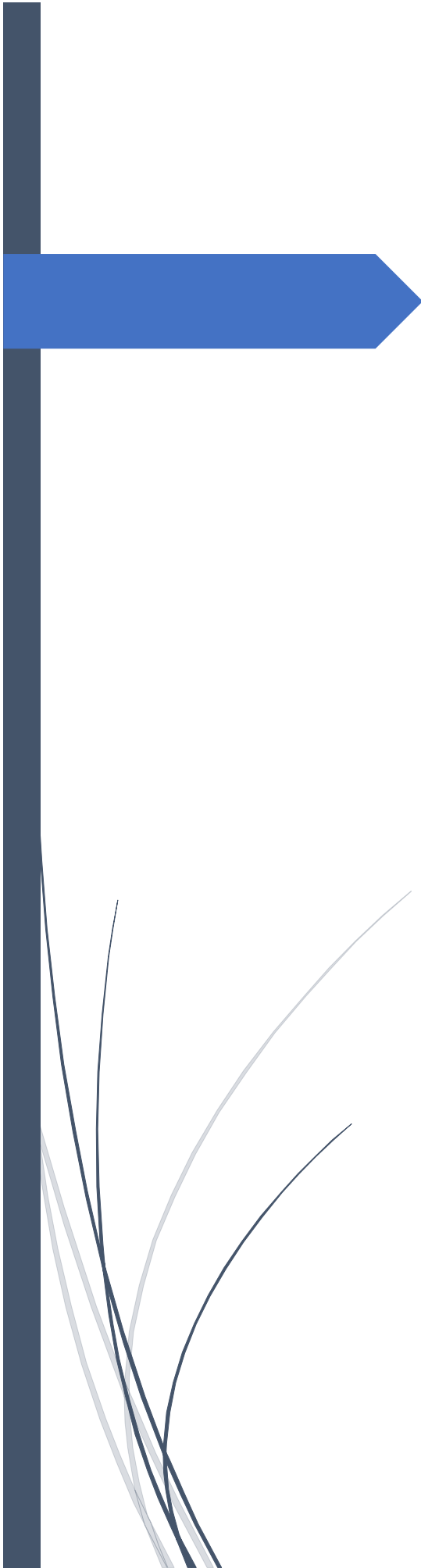




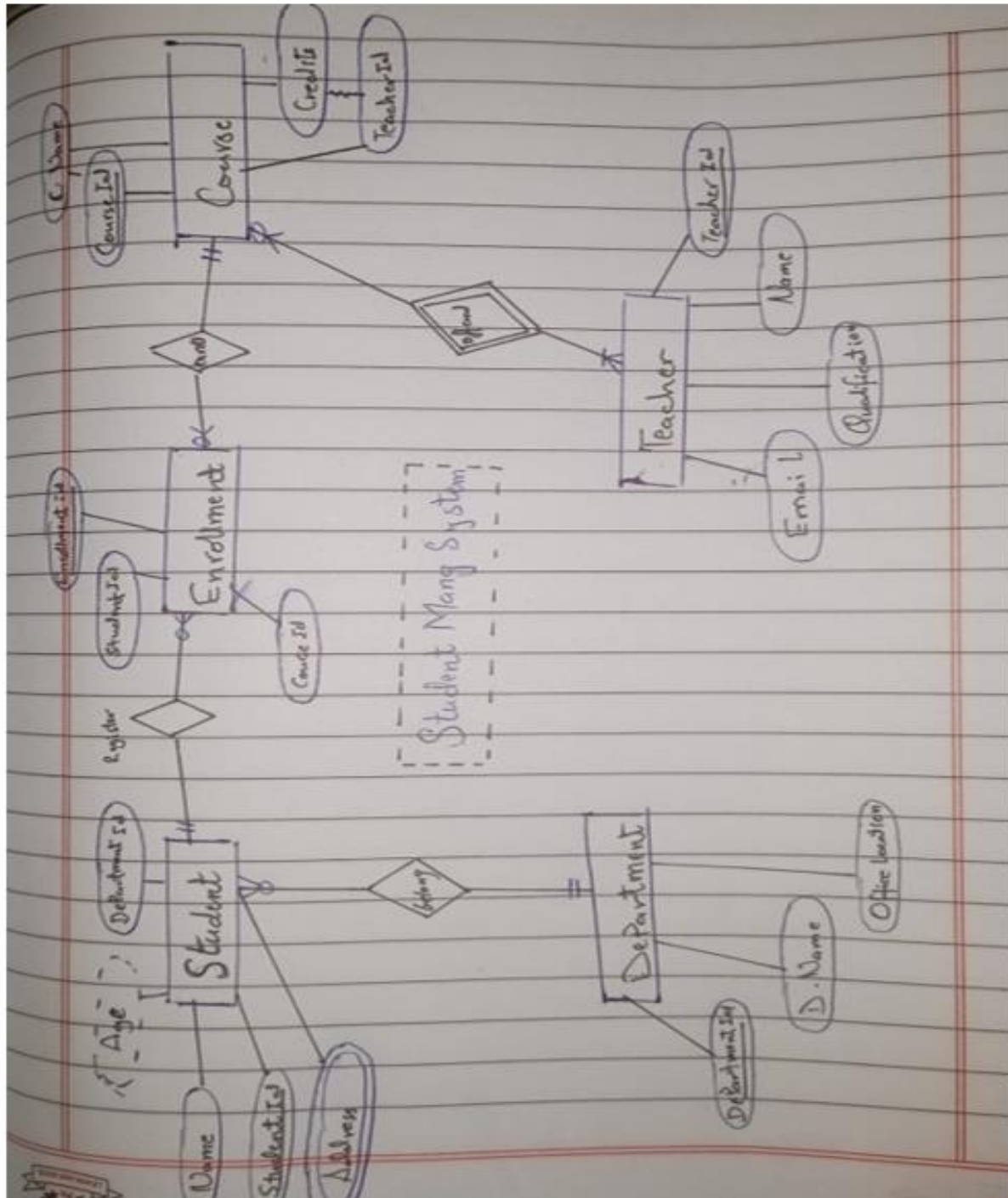
REPORT

Student Record System

Hanzalla Baqir (65072)

Entity Relationship Diagram :-

Student Record Management System





1. Core Schema Objects

This section describes the foundational elements of the StudentRecordDB database.

1.1 Tables and Fields

Department

- Department_ID (INT, Primary Key)
 - Department_Name (VARCHAR(100))
 - Office_Location (VARCHAR(100))
- Purpose: Stores information about university departments.

Student

- Student_ID (INT, Primary Key)
 - Name (VARCHAR(100))
 - Age (INT)
 - Class (VARCHAR(50))
 - Address (VARCHAR(255))
 - Department_ID (INT, Foreign Key referencing Department)
- Purpose: Stores student personal and academic information.

Teacher

- Teacher_ID (INT, Primary Key)
 - Name (VARCHAR(100))
 - Email (VARCHAR(100))
 - Department_ID (INT, Foreign Key referencing Department)
- Purpose: Stores teacher information and their department affiliation.

Course

- Course_ID (INT, Primary Key)
- Course_Name (VARCHAR(100))
- Credits (INT)



Hanzalla Baqir (65072)

- Teacher_ID (INT, Foreign Key referencing Teacher)
Purpose: Stores course details and the assigned instructor.

Enrollment

- Enrollment_ID (INT, Primary Key)
 - Student_ID (INT, Foreign Key referencing Student)
 - Course_ID (INT, Foreign Key referencing Course)
 - Enrollment_Date (DATE)
Purpose: Resolves the many-to-many relationship between Student and Course.
-

1.2 Keys and Constraints

- Primary Keys: Every table has a primary key (e.g., Student_ID, Department_ID) with the AUTO_INCREMENT attribute to ensure each record is unique.
- Foreign Keys:
 - Student and Teacher tables reference Department via Department_ID. The ON DELETE SET NULL action ensures that if a department is removed, the students' and teachers' department link is removed without deleting their core records.
 - Course references Teacher via Teacher_ID, with the same ON DELETE SET NULL behavior.
 - Enrollment links the Student and Course tables. The ON DELETE CASCADE action ensures that enrollment records are automatically deleted if the associated student or course is removed.
- Check Constraints:
 - Student.Age: A CHECK constraint ensures the age is within the valid range (15 to 60).
 - Course.Credits: A CHECK constraint ensures that credits are a positive integer (>0).
- Unique Constraints:



Hanzalla Baqir (65072)

- Department.Department_Name: Ensures no two departments can have the same name.
- Teacher.Email: Ensures all teacher email addresses are unique.
- Enrollment (Student_ID, Course_ID): A composite unique key prevents a student from enrolling in the same course multiple times.

1.3 Normalization (Up to 3NF)

The database design has been validated to be in Third Normal Form (3NF):

1. 1NF: All tables have atomic values (no repeating groups). A composite primary key is used for the Enrollment table to ensure unique rows.
2. 2NF: There are no partial dependencies. In the Enrollment table, which has a single-column primary key, all non-key attributes (like Enrollment_Date) are fully dependent on the entire primary key.
3. 3NF: No transitive dependencies exist. For example, the Teacher table stores the Department_ID to link to the department, but does not store the Department_Name directly, preventing a dependency of a non-key attribute on another non-key attribute.

First Normal Form (1NF) :

1. **1NF:** All tables have atomic values (no repeating groups). A composite primary key is used for the Enrollment table to ensure unique rows.

Second Normal Form (2NF) :

Rule: Meet all 1NF requirements AND remove partial dependencies. Non-key attributes must depend on the entire primary key.

Identified Partial Dependencies:

- Student_Name, Age, Class, Address, Dept_ID, Dept_Name, Office_Location depend only on Student_ID



Hanzalla Baqir (65072)

- Course_Name, Credits, Teacher_ID, Teacher_Name, Teacher_Email depend only on Course_ID
- Only Enrollment_Date depends on the full key (Student_ID, Course_ID)

Decomposition into 2NF Tables:

Table A: Student Table

Primary Key: Student_ID

ID	Name	Age	Class	Address	D.Id	D.Name	Office location
1	Hanzala Baqir	21	CS-4	Rwp	1	Computer Science	Block A
2	Ahmed Ali	22	CS-4	Islamabad	1	Computer Science	Block A
3	Usman Khan	23	SE-3	Lahore	2	Electrical Engineering	Block B

Table B: Course Table

Primary Key: Course_ID

Course Id	Course Name	Credits	Teacher Id	Teacher Name	T. Email
1	DataBase	3	1	Ali Khan	ali@uni.com
2	Electrical Circuits	4	2	Sara Ahmed	sara@uni.com
3	Marketing	3	3	John Smith	john@uni.com

Table C: Enrollment Table

Primary Key: Composite (Student_ID, Course_ID)

Student Id	Course Id	Enrollment Date
1	1	2026-01-15
1	3	2026-01-20



Hanzalla Baqir (65072)

2	1	2026-01-15
3	2	2026-02-10

Third Normal Form (3NF) :

Meet all 2NF requirements AND remove transitive dependencies. No non-key attribute should depend on another non-key attribute.

Identified Transitive Dependencies:

- In the Student Table: Dept_Name and Office_Location depend on Dept_ID (a non-key attribute)
- In the Course Table: Teacher_Name and Teacher_Email depend on Teacher_ID (a non-key attribute)

Decomposition into 3NF Tables (Final Schema):

Department :

Primary Key: Department_ID

Department Id	Department Name	Office Location
1	Computer Science	Block A
2	Electrical Engineering	Block B
3	Business Administration	Block C

Student :

Primary Key: Student_ID

Foreign Key: Department_ID → Department(Department_ID)

Student Id	Name	Age	Class	Address	Department Id
1	Hanzala Baqir	21	CS-4	Rwp	1
2	Ahmed Ali	22	CS-4	Islamabad	1
3	Usman Khan	23	SE-3	Lahore	2
4	Ayesha Noor	20	BBA-2	Karachi	3



Hanzalla Baqir (65072)

Teacher :

Primary Key: Teacher_ID

Foreign Key: Department_ID → Department(Department_ID)

Teacher Id	Teacher Name	Email	Department Id
1	Ali Khan	ali@uni.com	1
2	Sara Khan	sara@uni.com	2
3	John Smith	john@uni.com	3

Course:

Primary Key: Course_ID

Foreign Key: Teacher_ID → Teacher(Teacher_ID)

Course Id	Course Name	Credits	Teacher Id
1	DataBase	3	1
2	Electrical Circuits	4	2
3	Marketing	3	3

Enrollment

Primary Key: Composite (Student_ID, Course_ID)

Foreign Key: Student_ID → Student(Student_ID)

Foreign Key: Course_ID → Course(Course_ID)

Student Id	Course Id	Enrollment Date
1	1	2026-01-15
1	3	2026-01-20
2	1	2026-01-15
3	2	2026-02-10
4	3	2026-03-05



2. Queries and Implementation

This section details the SQL queries used to retrieve and manipulate data, as implemented in the provided script.

2.1 Joins

-- Query 1: Retrieve Student details along with their Department Name (LEFT JOIN to include students without a department)

```
SELECT s.Student_ID, s.Name, s.Class, d.Department_Name
FROM Student s
LEFT JOIN Department d ON s.Department_ID = d.Department_ID;
```

-- Query 2: Retrieve Student names and the Courses they are enrolled in (Inner Join through the Enrollment junction table)

```
SELECT s.Name AS Student, c.Course_Name
FROM Enrollment e
JOIN Student s ON e.Student_ID = s.Student_ID
JOIN Course c ON e.Course_ID = c.Course_ID;
```

-- Query 3: Retrieve Teacher names and the Courses they teach

```
SELECT t.Name AS Teacher, c.Course_Name
FROM Course c
JOIN Teacher t ON c.Teacher_ID = t.Teacher_ID;
```

2.2 Aggregation Functions

-- Query 4: Count the total number of students in each department

```
SELECT d.Department_Name, COUNT(s.Student_ID) AS Total_Students
FROM Department d
LEFT JOIN Student s ON d.Department_ID = s.Department_ID
GROUP BY d.Department_Name;
```

-- Query 5: Count the total number of students in the entire database

```
SELECT COUNT(*) AS Total_Students FROM Student;
```

-- Query 6: Calculate the average age of all students

```
SELECT AVG(Age) AS Average_Age FROM Student;
```



2.3 Subqueries

Although not explicitly written as a separate statement in the provided code, the logic for finding specific data points (like students in a specific department) can be efficiently performed with subqueries. A typical use case in this schema would be to find students enrolled in courses taught by a specific teacher.

-- Example: Find students enrolled in courses taught by 'Ali Khan'

```
SELECT Name
FROM Student
WHERE Student_ID IN (
    SELECT e.Student_ID
    FROM Enrollment e
    JOIN Course c ON e.Course_ID = c.Course_ID
    JOIN Teacher t ON c.Teacher_ID = t.Teacher_ID
    WHERE t.Name = 'Ali Khan'
);
```

2.4 Views

- **View Name:** StudentDepartmentView
- **Purpose:** To provide a simplified, reusable query that shows student information along with their department name without needing to write the join each time.

-- Create the view (Not in original script but recommended for report)

```
CREATE VIEW StudentDepartmentView AS
SELECT
    s.Student_ID,
    s.Name,
    s.Class,
    d.Department_Name
FROM Student s
LEFT JOIN Department d ON s.Department_ID = d.Department_ID;
```

-- Query the view

```
SELECT * FROM StudentDepartmentView;
```



Hanzalla Baqir (65072)

GitHub Repository :

System <https://github.com/HanzallaBaqir/Student-Record-System>